8086 CPU Overview

chatgpt.com/c/69b8e0f7-6a4c-8321-9483-1c982ccc1879

Chapter 1: Microcomputer Systems

1.1.2 The CPU (Central Processing Unit)

The **CPU** is the **brain of a computer**. It executes all instructions and controls system operations.

- Each instruction is a **binary bit string**
- In **8086**, instruction length = **1 to 6 bytes**
- The complete set of instructions is called the **Instruction Set**

Internal Organization of 8086

The **8086 microprocessor** has two main units:

1. Execution Unit (EU)

2. Bus Interface Unit (BIU)

Also:

- It is a **16-bit processor**
  - ALU works with 16-bit data
  - Registers are mostly 16-bit
- Data Bus = **16-bit**
- Address Bus = **20-bit**

  Can access **2^{20} = 1 MB memory**

1. Execution Unit (EU)

This unit **executes instructions**.

Main Components

1. ALU (Arithmetic Logic Unit)

Performs:

- Arithmetic: +, −, ×, ÷
- Logical: AND, OR, NOT

2. General Purpose Registers (8 total)

Register	Purpose
AX	Accumulator
BX	Base
CX	Count
DX	Data
SI	Source Index
DI	Destination Index
BP	Base Pointer
SP	Stack Pointer

3. FLAG Register

- Stores status of operations
- Indicates result conditions (zero, carry, sign, etc.)

4. Temporary Registers

Hold intermediate data for ALU operations

2. Bus Interface Unit (BIU)

This unit **connects CPU with memory and I/O devices**.

Functions

- Sends/receives:
  - Address signals
  - Data
  - Control signals
- Handles memory communication

Registers in BIU (5 total)

Register	Purpose
CS	Code Segment
DS	Data Segment
SS	Stack Segment
ES	Extra Segment
IP	Instruction Pointer

1.2 Instruction Execution

Instruction Format

Each instruction has:

- **Opcode** → what operation to perform
- **Operand** → data / register / memory address

Fetch-Execute Cycle

CPU executes instructions in steps:

Fetch Phase

1. Fetch instruction from memory
2. Decode instruction (understand operation)
3. Fetch operand (if needed)

Execute Phase

1. Perform operation

2. Store result (if required)

1.5 Assembly Language Program (8086)

Goal

Add two numbers stored in memory (A and B) and store result in SUM

Program Explanation (Step-by-step)

```
.MODEL SMALL
```

Defines memory model

```
.STACK 100H
```

Allocates stack (256 bytes)

```
.DATA
A DW 2
B DW 5
SUM DW ?
```

Declares variables:

- A = 2
- B = 5
- SUM = result (uninitialized)

```
.CODE
MAIN PROC
```

Start of program

Initialize Data Segment

```
MOV AX, @DATA
MOV DS, AX
```

Perform Addition

```
MOV AX, A     ; Load A into AX
ADD AX, B     ; AX = AX + B
MOV SUM, AX   ; Store result in SUM
```

Program Exit

```
MOV AX, 4C00H
INT 21H
```

Terminates program

```
MAIN ENDP
END MAIN
```

Chapter 2: Microprocessor Architecture and Microcomputer System

Microprocessor

A **microprocessor** is a:

- **Programmable logic device**
- Built using **LSI / VLSI technology**
- Contains:
  - Registers
  - Flip-flops
  - Timing/control elements

Key Idea

It is a **single chip CPU** of a microcomputer system

Microprocessor Unit (MPU)

- In early systems, CPU components were **not on a single chip**
- Instead, they were built using **multiple chips**
- This group of chips is called:

Microprocessor Unit (MPU)

2.1 Microprocessor Architecture and Operations

Architecture of Microprocessor

The **internal structure (logic design)** of a microprocessor is called its **architecture**

Categories of Operations

All functions of a microprocessor are divided into **3 main types**:

1. Microprocessor-Initiated Operations

2. Internal Data Operations

3. Peripheral (Externally) Initiated Operations

2.1.1 Microprocessor-Initiated Operations

These are operations **directly controlled by the microprocessor**.

Main Operations

1. **Memory Read**

   Reads data from memory

2. **Memory Write**

   Writes data to memory

3. **I/O Read**

   Takes input from devices

4. **I/O Write**

   Sends data to output devices

Steps to Communicate with Memory or Peripheral

To interact with memory or I/O devices:

1. **Identify address**

   Select memory location or device

2. **Transfer data**

   Send or receive data

3. **Provide timing signals**

   Synchronize operation

2.1.2 Internal Data Operations (8085/8080A)

These operations happen **inside the microprocessor**.

Capabilities

- Store **8-bit data**
- Perform:
  - Arithmetic operations
  - Logical operations
- Test conditions (e.g., zero, carry)
- Control execution sequence
- Use **stack** for temporary storage

Stack

- A **temporary memory area**
- Used during:
  - Function calls
  - Interrupts
  - Intermediate storage

2.1.3 Peripheral (Externally) Initiated Operations

These operations are **triggered by external signals or devices**.

1. Reset

- Stops all operations
- Clears **Program Counter (PC = 0000H)**
- Restarts program execution

2. Interrupt

- Temporarily **stops current program**
- Executes a **special routine (Interrupt Service Routine)**
- Then returns to original program

3. Ready

- Used for **synchronization with slow devices**
- If READY = LOW:

  CPU enters **Wait State**

4. Hold

- External device requests control of system bus
- CPU **releases bus control**

Example:

Used in **DMA (Direct Memory Access)**

Chapter 3: Organization of the IBM Personal Computers

3.2 Organization of the 8086/8088 Microprocessors

3.2.1 Registers Overview

Registers are classified based on their function:

- **Data Registers** → store data
- **Address Registers** → store memory addresses
- **Status Register** → shows processor condition

Total Registers in 8086

- **14 registers**
- All are **16-bit**

3.2.2 Data Registers

Registers

AX, BX, CX, DX

Each register can be split into:

- **High byte**: AH, BH, CH, DH
- **Low byte**: AL, BL, CL, DL

Why split?

Helps handle **8-bit data efficiently**

Functions of Each Register

AX (Accumulator)

- Used in:
  - Arithmetic operations
  - Logic operations
  - Data transfer
- Required in:
  - Multiplication & division
  - Input/output operations

BX (Base Register)

Used as:

- Data register
- Address register

CX (Count Register)

- Used as:

  Loop counter

- Important for:
  - Loops
  - Shift & rotate
  - String operations

DX (Data Register)

Used in:

- Multiplication & division
- I/O operations

3.2.3 Segment Registers

Registers

- CS (Code Segment)
- DS (Data Segment)
- SS (Stack Segment)
- ES (Extra Segment)

Function

Store **starting address of memory segments**

Important Facts

- Address bus = **20-bit → 1 MB memory**
- Each segment size = **64 KB (2^{16} bytes)**
- At a time CPU works with **4 segments**

Memory Segmentation Concept

Segment

A block of **64 KB continuous memory**

Offset

- Distance from start of segment
- Range: **0000H to FFFFH**

Segment : Offset Address (Logical Address)

Format:

```
Segment : Offset
```

Example:

```
A4FB : 4872
```

Physical Address Calculation

To get **20-bit physical address**:

- Shift segment left by 4 bits (×10H)
- Add offset

Formula

Physical Address = (Segment × 10H) + Offset

Example

```
Segment = A4FBH
Offset  = 4872H
```

```
A4FB0H
+4872H
= A9822H
```

Segment Overlapping

- Segments start every **16 bytes (10H)**
- This 16-byte block is called a **Paragraph**
- Segments can **overlap**

Important

- Segment addresses always end with **0**
- Multiple segment:offset pairs can give **same physical address**

3.2.4 Pointer and Index Registers

Registers

- SP (Stack Pointer)
- BP (Base Pointer)
- SI (Source Index)

- DI (Destination Index)

Functions

SP (Stack Pointer)

- Points to top of stack
- Works with **SS**

BP (Base Pointer)

- Access data from stack
- Can also access other segments

SI (Source Index)

- Points to data in **data segment (DS)**
- Used for sequential access

DI (Destination Index)

- Similar to SI
- Used in **string operations**
- Works with **extra segment (ES)**

Instruction Pointer (IP)

- Holds **offset of next instruction**
- Works with **CS register**

Instruction Address

```
CS : IP
```

Automatically updates after execution

3.2.6 Flags Register

- A **16-bit register**
- Contains **9 active flags**

Types of Flags

1. Status Flags (6)

Indicate result of operations

Examples:

- **ZF (Zero Flag)** → result = 0
- **CF (Carry Flag)** → carry generated
- Others: Sign, Overflow, etc.

2. Control Flags (3)

Control processor behavior

Examples:

- **IF (Interrupt Flag)** → enable/disable interrupts
- Used in:
  - Interrupt handling
  - String operations

Chapter 4: Introduction to IBM PC Assembly Language

4.1 Assembly Language Syntax

- Assembly programs are translated into machine code by an **assembler**
- Code is **not case-sensitive** (but uppercase is commonly used)

Structure of a Statement

Each statement has **4 fields**:

```
Name   Operation   Operand(s)   Comment
```

Example

```
START: MOV CX, 5 ; initialize counter
```

- **Name** → START
- **Operation** → MOV
- **Operands** → CX, 5
- **Comment** → explanation

Types of Statements

1. Instruction

- Translated into machine code
- Example: `MOV AX, BX`

2. Assembler Directive (Pseudo-op)

- Not converted to machine code
- Used to guide assembler
- Example: `PROC`, `.DATA`, `.CODE`

4.1.1 Name Field

- Used for:
  - Labels
  - Variables
  - Procedures
- Rules:
  - Length: **1–31 characters**
  - Must start with a **letter**
  - Can contain digits & symbols

4.1.2 Operation Field

1. Opcode

- Real instruction
- Example: `MOV`, `ADD`, `SUB`

2. Pseudo-op

- Assembler directive
- Example: `PROC`, `DB`, `DW`

4.1.3 Operand Field

- Specifies data to operate on
- Can be:
  - Register
  - Memory
  - Immediate (constant)

Rules

- Max **2 operands**
- Destination **cannot be constant**
- Memory-to-memory **not allowed** (except special cases)

4.1.4 Comment Field

- Starts with `;`
- Used for explanation
- Ignored by assembler

4.2 Program Data

Number Formats

Type	Format
Binary	ends with B
Decimal	optional D
Hex	ends with H (must start with digit)

Example:

- `1101B`
- `25D`
- `0FFFFH`

Characters

- Written in quotes: `'A'`, `"HELLO"`
- Converted to **ASCII automatically**

Data Definition Pseudo-ops

Pseudo-op	Meaning
DB	Define Byte
DW	Define Word
DD	Double word
DQ	Quad word
DT	Ten bytes

4.3 Variables

Byte Variable

```
name DB value
```

Example:

```
A DB 5
B DB ?
```

Word Variable

```
name DW value
```

Example:

```
X DW 100
Y DW ?
```

Arrays

Sequence of bytes/words

Example:

```
ARR DB 10H, 20H, 30H
```

Access:

- ARR → first element
- ARR+1 → second
- ARR+2 → third

Character Strings

```
MSG DB 'HELLO'
```

Equivalent:

```
MSG DB 48H, 45H, 4CH, 4CH, 4FH
```

4.4 Named Constants (EQU)

```
name EQU value
```

Example:

```
LF EQU 0AH
```

- No memory allocated
- Just a **symbolic name**

4.5 Basic Instructions

MOV Instruction

Transfers data

```
MOV destination, source
```

Examples:

```
MOV AX, BX
MOV AX, WORD1
MOV AH, 'A'
```

XCHG Instruction

Swaps data

```
XCHG AX, BX
```

Restriction

Memory-to-memory **not allowed**

ADD & SUB

```
ADD destination, source
SUB destination, source
```

Example:

```
ADD AX, BX
SUB AX, DX
```

INC & DEC

```
INC AX
DEC AX
```

Increase/decrease by 1

NEG Instruction

```
NEG AX
```

Converts to **2's complement**

Type Rule

Both operands must be same type:

- Byte ↔ Byte
- Word ↔ Word

4.7 Program Structure

Memory Model

```
.MODEL SMALL
```

Common Models

Model	Description
SMALL	1 code, 1 data
MEDIUM	many code, 1 data
COMPACT	1 code, many data
LARGE	many code, many data

Segments

1. Data Segment

```
.DATA
A DW 5
```

2. Stack Segment

```
.STACK 100H
```

3. Code Segment

```
.CODE
MAIN PROC
...
MAIN ENDP
```

Complete Structure

```
.MODEL SMALL
.STACK 100H
.DATA
...
.CODE
MAIN PROC
...
MAIN ENDP
END MAIN
```

4.8 Input/Output Instructions

IN and OUT

- Direct port communication
- Rarely used in high-level programs

INT Instruction

```
INT number
```

Calls system service

INT 21H (DOS Interrupt)

Function 1 → Input

```
MOV AH, 1
INT 21H
```

Input char → stored in AL

Function 2 → Output

```
MOV AH, 2
MOV DL, 'A'
INT 21H
```

Function 9 → Print String

```
MOV AH, 9
LEA DX, MSG
INT 21H
```

String must end with `$`

Control Characters

Code	Function
07H	Beep
08H	Backspace
0AH	New line
0DH	Carriage return

LEA Instruction

```
LEA DX, MSG
```

Loads **address** (not value)

Program Segment Prefix (PSP)

- DOS loads program with **256-byte PSP**
- DS initially points to PSP

Fix

```
MOV AX, @DATA
MOV DS, AX
```

Example Programs

1. Echo Program (Input + Output)

- Reads a character
- Prints it on next line

2. String Display Program

```
MSG DB 'HELLO$'
```

```
MOV AX, @DATA
MOV DS, AX
LEA DX, MSG
MOV AH, 9
INT 21H
```

Chapter 5: Processor Status and the FLAGS Register

Overview

- The **8086 processor** can make decisions based on its **current state**
- This state is stored in **FLAGS register**
- FLAGS contains **9 important bits (flags)**

What is a Flag?

- A **flag is a flip-flop**

- It:
    - Indicates result of an operation
    - Controls processor behavior

5.1 FLAGS Register

- Total size = **16-bit**
- Active flags = **9**
- Divided into:
    - **Status Flags (6)**
    - **Control Flags (3)**

Status Flags (6)

These indicate **result of arithmetic/logical operations**:

1. Carry Flag (CF)

- CF = 1 → carry out (addition) or borrow (subtraction)
- Used in:
    - Arithmetic
    - Shift/rotate

2. Parity Flag (PF)

- PF = 1 → even number of 1s in **low byte**
- PF = 0 → odd number of 1s

3. Auxiliary Carry Flag (AF)

- AF = 1 → carry from bit 3 to bit 4
- Used in:

    BCD operations

4. Zero Flag (ZF)

- ZF = 1 → result = 0
- ZF = 0 → result ≠ 0

5. Sign Flag (SF)

- SF = 1 → result is negative (MSB = 1)
- SF = 0 → positive

6. Overflow Flag (OF)

- OF = 1 → **signed overflow occurred**
- OF = 0 → no overflow

Control Flags (3)

These control processor operations:

1. Trap Flag (TF)

Used for **debugging (single-step execution)**

2. Interrupt Flag (IF)

- IF = 1 → interrupts enabled
- IF = 0 → interrupts disabled

3. Direction Flag (DF)

- Used in **string operations**
- Controls direction:

  Forward or backward

5.2 Overflow

Types of Overflow

There are **4 possible cases**:

1. No overflow
2. Signed overflow only
3. Unsigned overflow only
4. Both

Unsigned Overflow

Occurs when:

> Result exceeds **maximum unsigned value**

Condition

> Carry out of MSB → **CF = 1**

Example

```
FFFFH + 0001H = 10000H
```

- Stored result = 0000H (wrong)
- CF = 1 → unsigned overflow

Signed Overflow

Occurs when:

> Result sign is incorrect

Condition

> OF = 1

Rules

- Adding **same sign numbers**:

  > If result sign changes → overflow

- Adding **different signs**:

  > No overflow possible

Example

```
7FFFH + 7FFFH = FFFEH
```

- Positive + Positive → Negative
- OF = 1 → signed overflow

How Processor Detects Overflow

- **CF = 1 → unsigned overflow**
- **OF = 1 → signed overflow**

Processor does:

- Checks both cases automatically
- Program decides what to do

5.3 Effect of Instructions on Flags

Instruction	Effect
MOV	No flags affected
XCHG	No flags affected
ADD/SUB	All flags affected
INC/DEC	All except CF
NEG	All affected

NEG Special Case

- CF = 1 (if result ≠ 0)
- OF = 1 if:
    - Word = 8000H
    - Byte = 80H

Example (Important)

Instruction

```
ADD AX, BX
AX = FFFFH
BX = FFFFH
```

Result

```
FFFFH + FFFFH = 1FFFEH
Stored = FFFEH
```

Flags

- **SF = 1** → negative
- **PF = 0** → odd parity
- **ZF = 0** → not zero
- **CF = 1** → carry occurred
- **OF = 0** → no signed overflow

Chapter 6: Flow Control Instructions

Overview

- Flow control instructions are used to:
  - **Make decisions**
  - **Repeat code (loops)**
- They transfer control to another part of the program

Types

- **Conditional** (depends on FLAGS)
- **Unconditional** (always jump)

6.1 Example of Jump (Loop)

Uses:

- `JNZ` (jump if not zero)
- `CX` as counter

Concept

Repeat loop until **CX = 0**

6.2 Conditional Jumps

Syntax

```
Jxxx label
```

Jump occurs **only if condition is true**

How It Works

- CPU checks **FLAGS register**

- If condition is true:

  IP jumps to label

Range Limitation

Jump range:

-126 to +127 bytes

CMP Instruction

Syntax

```
CMP destination, source
```

- Performs: `destination - source`
- Does **not store result**
- Only **affects FLAGS**

Types of Conditional Jumps

1. Signed Jumps

Used when numbers are **signed**

Instruction	Condition
JG	greater (ZF=0 and SF=OF)
JGE	greater or equal (SF=OF)
JL	less (SF ≠ OF)
JLE	less or equal (ZF=1 or SF ≠ OF)

2. Unsigned Jumps

Used when numbers are **unsigned**

Instruction	Condition
JA	above (CF=0 and ZF=0)
JAE	above or equal (CF=0)
JB	below (CF=1)
JBE	below or equal (CF=1 or ZF=1)

3. Single-Flag Jumps

Instruction	Condition
JE / JZ	ZF = 1
JNE / JNZ	ZF = 0
JC	CF = 1
JNC	CF = 0
JO	OF = 1
JNO	OF = 0
JS	SF = 1
JNS	SF = 0
JP	PF = 1
JNP	PF = 0

Important Notes

- Jump instructions **do NOT change flags**
- CMP is usually used **before jumps**

Signed vs Unsigned (Important Concept)

Same values → different results depending on interpretation

Example

- AX = 7FFFH

- BX = 8000H
- Signed → AX > BX
- Unsigned → AX < BX

6.3 JMP Instruction (Unconditional Jump)

Syntax

```
JMP label
```

- Always jumps
- No condition check

Use Case

Avoid range limitation of conditional jumps

6.4 High-Level Structures in Assembly

1. IF-THEN

Concept

```
IF condition
   statements
```

Example (Absolute Value)

```
CMP AX, 0
JNL END_IF
NEG AX
END_IF:
```

2. IF-THEN-ELSE

Structure

```
IF condition
   true-part
ELSE
   false-part
```

3. CASE Structure

Multiple conditions

Example Idea

- AX < 0 → BX = -1
- AX = 0 → BX = 0
- AX > 0 → BX = 1

4. Compound Conditions

AND Condition

Must satisfy **both conditions**

Example:

Check uppercase:

```
CMP AL, 'A'
JNGE END
CMP AL, 'Z'
JNLE END
```

OR Condition

Any one condition true

Example:

```
CMP AL, 'y'
JE THEN
CMP AL, 'Y'
JE THEN
```

6.4.2 Looping Structures

1. FOR Loop (Using LOOP)

Syntax

```
LOOP label
```

How it Works

- CX = loop counter
- LOOP:
    - Decrement CX
    - If CX ≠ 0 → jump

Example

```
MOV CX, 80
TOP:
; body
LOOP TOP
```

Important Issue

If CX = 0 initially → loop runs **65535 times**

Solution

```
JCXZ SKIP
```

2. WHILE Loop

Concept

```
WHILE condition
   statements
```

Example

Count characters until ENTER:

```
CMP AL, 0DH
JE END
INC DX
JMP LOOP
```

Chapter 7: Logic, Shift, and Rotate Instructions (Conceptual Note)

7.1 Logic Instructions (Bit Manipulation)

Core Idea

Logic instructions work at the **bit level**.
Each bit of the operand is processed **individually**.

This is powerful because you can:

- Turn specific bits ON/OFF
- Check conditions
- Modify data efficiently

AND, OR, XOR (Most Important)

1. AND Instruction

- Used to **clear (make 0)** specific bits
- Rule:
  - `0 → clears bit`
  - `1 → keeps original`

Use case: Masking unwanted bits

Example idea:

> Want to remove sign bit → use AND mask

2. OR Instruction

- Used to **set (make 1)** specific bits
- Rule:
  - `1 → sets bit`
  - `0 → keeps original`

Use case: Enable certain flags/bits

3. XOR Instruction

- Used to **toggle (flip)** bits
- Rule:
  - `1 → flips bit`
  - `0 → keeps original`

Special use:

> `XOR AX, AX` → makes AX = 0 (very efficient)

Mask Concept (Very Important)

A **mask** is a bit pattern used to control which bits change.

Operation	Purpose
AND mask	Clear bits
OR mask	Set bits
XOR mask	Toggle bits

Other Logic Instructions

NOT

- Flips all bits (1 ↔ 0)
- No flag changes

TEST

- Performs AND **but does NOT store result**
- Only updates flags

Use: Check bits without modifying data

Example concept:

> Check even/odd:
>
> > If LSB = 0 → even

Important Applications

1. ASCII to Number

- ASCII ‘0’ = 30H
- So subtract 30H to get number

2. Lowercase → Uppercase

- Difference is only **one bit (bit 5)**
- Clearing that bit converts case

3. Clearing Register

Three ways:

- MOV AX, 0
- SUB AX, AX
- XOR AX, AX (best)

7.2 Shift Instructions

Core Idea

Shift = move bits left or right

Difference:

- **Shift → bits lost**
- **Rotate → bits reused**

Left Shift (SHL / SAL)

Concept

- Bits move left
- Rightmost bit becomes 0
- Leftmost bit goes to Carry Flag

Key Insight

Left shift = **Multiply by 2**

Example:

> 5 → 10 → 20 (each shift doubles)

SAL vs SHL

- Both are **same in practice**
- SAL used in arithmetic context

Right Shift

1. SHR (Logical Right Shift)

- MSB filled with 0
- Used for **unsigned numbers**

2. SAR (Arithmetic Right Shift)

- MSB **keeps original sign**
- Used for **signed numbers**

Key Insight

Right shift = **Divide by 2**

- Even numbers → exact
- Odd numbers → rounded down

Important Concept: Signed vs Unsigned

Operation	Use
SHR	Unsigned
SAR	Signed

7.3 Rotate Instructions

Core Idea

Rotate = circular shift
Bits are **not lost**, they come back

1. ROL (Rotate Left)

- Leftmost bit → goes to rightmost
- Also stored in Carry Flag

2. ROR (Rotate Right)

Rightmost bit → goes to leftmost

3. RCL / RCR (Rotate through Carry)

Key Difference

Carry flag becomes part of rotation

This creates a **larger circular shift including CF**

Important Application

Counting number of 1s

- Rotate bits one by one
- Count how many times Carry = 1

Bit Reversal

Use combination of:

Shift + Rotate

Summary (Very Important for Exam)

Logic Instructions

- AND → clear bits
- OR → set bits
- XOR → toggle bits
- TEST → check bits

Shift Instructions

- SHL → multiply by 2
- SHR → divide by 2 (unsigned)

- SAR → divide by 2 (signed)

Rotate Instructions

- ROL / ROR → circular shift
- RCL / RCR → circular with carry

Chapter 8: The Stack and Procedures (Conceptual Note)

Overview

- The **stack** is used for:
    - Temporary data storage
    - Saving return addresses
    - Supporting **procedure calls**
- It is essential for **structured programming in assembly**

8.1 The Stack

Core Concept

A stack is a **LIFO (Last-In, First-Out)** structure.

- Last data inserted → first removed
- Think of it like a stack of plates

Key Registers

- **SS (Stack Segment)** → points to stack memory
- **SP (Stack Pointer)** → points to top of stack

Stack Initialization

Example:

```
.STACK 100H
```

- Allocates stack memory
- SP = 100H initially → means **empty stack**

PUSH Operation

Concept

Adds data to the stack

Steps

1. SP = SP - 2
2. Store data at address SS:SP

Important Points

- Works only with **16-bit data**
- Source remains unchanged

POP Operation

Concept

Removes data from stack

Steps

1. Read data from SS:SP
2. SP = SP + 2

Important Rules

- Stack works in **word (2 bytes)** units

- Invalid:
    - `PUSH DL` (8-bit not allowed)
    - `PUSH 5` (immediate not allowed in older CPUs)

Special Instructions

- **PUSHF** → pushes FLAGS register
- **POPF** → restores FLAGS

Key Insight

Operation	Effect on SP
PUSH	SP decreases
POP	SP increases

8.2 Stack Application (Important Concept)

Example: Reverse Input

Idea:

1. Take input characters
2. PUSH each character into stack
3. POP them back → reversed output

Why it works?

Because stack is **LIFO**

8.3 Procedures (Functions in Assembly)

Concept

A **procedure** is a block of code that:

- Performs a specific task
- Can be reused

Structure

```
name PROC
   ; instructions
   RET
name ENDP
```

Types of Procedures

1. NEAR Procedure

- Call within same segment
- Faster

2. FAR Procedure

- Call from different segment
- Requires more memory

Main Procedure

- Entry point of program
- Execution starts here

Procedure Execution Flow

1. CALL procedure
2. Execute procedure code
3. RET → return back

8.4 CALL and RET

CALL Instruction

What happens internally?

1. Return address is pushed into stack
2. Control jumps to procedure

RET Instruction

What happens?

1. Address is popped from stack
2. Execution returns to caller

Key Insight

Stack is used automatically for:

- Saving return address
- Managing program flow

Direct vs Indirect CALL

Type	Description
Direct	CALL procedure_name
Indirect	CALL register/memory

RET with Argument

```
RET  4
```

- Pops return address

- Also removes extra 4 bytes from stack

Communication Between Procedures

Assembly has **no parameter system like C/C++**

So we use:

- Registers (AX, BX, etc.)
- Memory
- Stack

8.5 Example Concept: Multiplication Using Procedure

Idea

Multiply using:

- Addition
- Shift operations

Logic

- Check bit of multiplier
- Add if needed
- Shift values
- Repeat

Important Concept Used

- `TEST` → check bit
- `SHL` → multiply by 2
- `SHR` → divide by 2

Chapter 9: Multiplication and Division Instructions (Conceptual Note)

Overview

- Earlier:
  - Shift left → multiply by 2
  - Shift right → divide by 2
- Now:

  Perform **general multiplication and division**

Key Concept

Operations are different for:

- **Unsigned numbers**
- **Signed numbers**

So we use different instructions:

Operation	Unsigned	Signed
Multiply	MUL	IMUL
Divide	DIV	IDIV

9.1 Multiplication Instructions (MUL & IMUL)

Core Idea

- One operand is **implicit (fixed)**
- Result is **larger than operands**

Byte Multiplication (8-bit)

- One operand → AL
- Other → source
- Result → AX (16-bit)

Word Multiplication (16-bit)

- One operand → AX

- Other → source
- Result → DX:AX (32-bit)

Important Insight

Input Size	Output Size
8-bit × 8-bit	16-bit
16-bit × 16-bit	32-bit

MUL vs IMUL

MUL (Unsigned)

Treats numbers as **positive only**

IMUL (Signed)

Handles **negative numbers**

Flags Behavior (Very Important)

CF and OF indicate overflow

For MUL

- CF = OF = 0 → upper part = 0
- CF = OF = 1 → overflow (result too large)

For IMUL

- CF = OF = 0 → valid signed result
- CF = OF = 1 → overflow

Key Concept

If result **does not fit** in:

- AL (for byte)
- AX (for word)

→ then overflow occurs

Important Insight

- If numbers are positive → MUL and IMUL give same result
- Difference appears when **negative numbers involved**

9.2 Applications of Multiplication

Example Concept: Expression Evaluation

High-level:

```
A = 5*A - 12*B
```

Idea:

1. Multiply A by 5
2. Multiply B by 12
3. Subtract

Factorial Example (Very Important)

Concept

Multiply numbers from 1 to N

Logic

- Start with AX = 1
- Loop:
    - Multiply by next number
    - Decrease counter

9.3 Division Instructions (DIV & IDIV)

Core Idea

- Dividend is **larger**
- Divisor is **smaller**
- Result = Quotient + Remainder

Byte Division (8-bit divisor)

- Dividend → AX (16-bit)
- Result:
  - Quotient → AL
  - Remainder → AH

Word Division (16-bit divisor)

- Dividend → DX:AX (32-bit)
- Result:
  - Quotient → AX
  - Remainder → DX

Important Insight

Operation	Registers Used
Byte Division	AX → AL + AH
Word Division	DX:AX → AX + DX

DIV vs IDIV

DIV (Unsigned)

Only positive numbers

IDIV (Signed)

Supports negative values

Important Rule

> Remainder has **same sign as dividend** (for signed division)

Flags Behavior

- All flags → **undefined**
- So flags are **not useful after division**

Divide Overflow (Very Important)

Occurs when:

> Quotient is too large to fit in destination

Example:

> Result doesn't fit in AL or AX

Key Differences (Must Remember)

Feature	MUL	DIV
Operation	Multiply	Divide
Result size	Larger	Same as divisor
Registers	AX / DX:AX	AX / DX
Flags	CF, OF used	Undefined

Chapter 10: Arrays and Addressing Modes

10.1 One-Dimensional Arrays

- **Definition:** A collection of elements of the same type stored consecutively in memory.
- **Declaration:**
  - Byte array: `MSG DB 'abcde'`
  - Word array: `W DW 10,20,30,40,50,60`
- **Base Address:** The starting memory location of the array.

- **Element Address:**

  For element N of size S bytes:
  `Address = Base + (N-1) × S`

DUP Operator

- Repeats a value to initialize arrays:
  - `GAMMA DW 100 DUP(0)` → 100 words initialized to 0
  - `DELTA DB 212 DUP(?)` → 212 uninitialized bytes
- **Nested DUPs** allowed:

  `LINE DB 5,4,3 DUP(2,3 DUP(0),1)` → expands into sequence

Example

Exchange 10th and 25th elements of a word array W:

```asm
MOV AX, W+18     ; W[10]
XCHG W+48, AX    ; W[25] ↔ AX
MOV W+18, AX
```

10.2 Addressing Modes

- **Purpose:** Specifies how to locate operands.
- **Previously used:**
  1. Register Mode → operand is register
  2. Immediate Mode → operand is constant
  3. Direct Mode → operand is variable

- **Additional 8086 modes for memory operands:**
    1. Register Indirect
    2. Based
    3. Indexed
    4. Based-Indexed

10.2.1 Register Indirect Mode

- **Format:** `[register]` → register holds offset
- **Registers:** BX, SI, DI → segment DS; BP → segment SS
- **Example:**

```asm
MOV AX, [SI]   ; loads value at DS:SI into AX
```

- **Legal instructions examples:**
    - `MOV BX, [BX]` → legal
    - `ADD [SI],[DI]` → illegal (memory to memory not allowed)
- **Array sum example:**

```asm
XOR AX, AX
LEA SI, W
MOV CX, 10
ADDNOS:
ADD AX, [SI]
ADD SI, 2
LOOP ADDNOS
```

10.2.2 Based and Indexed Modes

- **Idea:** Memory offset = register + displacement
- **Displacement:** constant, variable offset, or combination
- **Registers:**
  - Base → BX or BP
  - Index → SI or DI
- **Syntax variations:**

```asm
[register + displacement]   ; standard
[displacement + register]   ; equivalent
```

Examples:

```asm
MOV AX, W[BX]   ; element at W + BX
MOV AX, [SI+2]  ; element at SI + 2
```

- **Based Mode:** Uses BX or BP as register
- **Indexed Mode:** Uses SI or DI as register
- **Sum using based mode:**

```asm
XOR AX, AX
XOR BX, BX
MOV CX, 10
ADDNOS:
ADD AX, W[BX]
ADD BX, 2
LOOP ADDNOS
```

Practical Examples

1. **Uppercase conversion of a string (Indexed Mode):**

```asm
MOV CX, 17
XOR SI, SI
TOP:
CMP MSG[SI], ' '
JE NEXT
AND MSG[SI], 0DFH   ; clear bit 5 → uppercase
NEXT:
INC SI
LOOP TOP
```

2. **Accessing array elements:**

- `MOV AX, [ALPHA+BX]` → legal
- `MOV AX, [BX+AX]` → illegal (cannot use AX as source register)

Chapter 11: String Instructions

Overview

- Strings are byte or word arrays.
- String instructions allow **efficient array processing**:
  - Copy, move, or store strings
  - Search for bytes/words
  - Compare strings alphabetically
- These instructions generally use **register indirect addressing** with **SI** (source index) and **DI** (destination index).

11.1 Direction Flag (DF)

- **Purpose:** Controls the direction of string operations.

- **DF = 0:** SI & DI increment → process left to right (increasing memory)
- **DF = 1:** SI & DI decrement → process right to left (decreasing memory)

Instructions:

```asm
CLD ; Clear DF → left to right
STD ; Set DF → right to left
```

11.2 Moving a String

- **MOVSB** – Move byte from DS:SI → ES:DI
- **MOVSW** – Move word from DS:SI → ES:DI
- **Automatic update:** SI & DI increment/decrement by 1 (MOVSB) or 2 (MOVSW) according to DF
- **REP prefix:** Repeats instruction CX times

```asm
CLD
LEA SI, STRING1
LEA DI, STRING2
MOV CX, 5
REP MOVSB ; copy entire string
```

Example: Reverse copy:

```asm
LEA SI, STRING1+4
LEA DI, STRING2
STD
MOV CX, 5
REP MOVSB
```

PTR operator forces data type:

```asm
MOV AX, WORD PTR [Num] ; Load word from DWORD
```

11.3 Storing a String

- **STOSB** – Store AL to ES:DI (byte)
- **STOSW** – Store AX to ES:DI (word)
- Updates DI by 1 or 2 according to DF
- **Example:** Store two 'A's in STRING1

```asm
LEA DI, STRING1
CLD
MOV AL, 'A'
STOSB
STOSB
```

11.4 Loading a String

- **LODSB** – Load byte from DS:SI → AL
- **LODSW** – Load word from DS:SI → AX
- Updates SI by 1 or 2 according to DF
- **Example:** Load first two bytes

```asm
LEA SI, STRING1
CLD
LODSB
LODSB
```

11.5 Scanning a String

- **SCASB** – Compare AL with byte at ES:DI (word version SCASW uses AX)
- Sets flags according to result, updates DI according to DF

- **REPNE / REPNZ prefix**: Repeat until not equal or CX = 0

```asm
LEA DI, STRING1
MOV AL, 'B'
CLD
REPNE SCASB ; search for 'B'
```

11.6 Comparing Strings

- **CMPSB** – Subtract byte at ES:DI from DS:SI, set flags
- **CMPSW** – Subtract word at ES:DI from DS:SI, set flags
- Updates SI & DI according to DF
- **Prefixes:**

  REPE / REPZ – Repeat while equal (ZF=1)
- **Usage:** Compare two strings alphabetically

```asm
MOV CX, 10
LEA SI, STR1
LEA DI, STR2
CLD
REPE CMPSB
; Flags indicate first mismatch or string end
```

Example: Determine alphabetical order

```asm
JL STR1_FIRST ; STR1 < STR2
JG STR2_FIRST ; STR1 > STR2
MOV AX, 0     ; STR1 = STR2
```

Key Points

- **SI** = source pointer, **DI** = destination pointer
- **DF** controls left-to-right or right-to-left string processing
- **MOVS / STOS / LODS / SCAS / CMPS** → key string instructions
- **REP, REPE/REPZ, REPNE/REPNZ** → repeat string operations efficiently

Structure of an 8086 Assembly Language Program

Basic Program Skeleton

```asm
.MODEL memory_model       ; Declare memory model (usually SMALL)
.STACK stack_size         ; Declare stack segment
.DATA                     ; Optional data segment
; Program variables go here
.CODE                     ; Code segment
MAIN PROC
; Main procedure instructions
MAIN ENDP
; Subprocedures (user-defined) go here
END MAIN                  ; End of program
```

Key Points

1. **.MODEL** – Specifies memory model (SMALL, MEDIUM, LARGE, etc.).
2. **.STACK** – Mandatory for stack; 100H (256 bytes) is commonly used even if stack is not explicitly used.
3. **.DATA** – Optional; only needed if variables or constants are used.
4. **.CODE** – Mandatory; contains program instructions.
5. **MAIN PROC** – Entry point of the program. Only one main procedure per program.
6. **Subprocedures** – Optional; user-defined functions or routines.
7. **Comments** – Use `;` to indicate a comment.

Input/Output Operations

Operation	Register Used	Function No.	Interrupt No.
Input Single Char	AL	01H	21H

Operation	Register Used	Function No.	Interrupt No.
Output Single Char	DL	02H	21H
Output String	DX	09H	21H

Example 1: Display a Character

```asm
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV DL, 'A'      ; Character to display
    MOV AH, 02H      ; Function 2 = Display single character
    INT 21H          ; DOS interrupt
    MOV AH, 4CH      ; Function 4C = Exit to DOS
    INT 21H
MAIN ENDP
END MAIN
```

Explanation:

- `DL` contains the ASCII value of the character to display.
- `AH` contains the function number for the desired DOS operation.
- `INT 21H` invokes DOS interrupt for I/O.

Example 2: Input and Display a Character

```asm
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV DL, '?'     ; Prompt character
    MOV AH, 02H     ; Display character
    INT 21H
    MOV AH, 01H     ; Input character
    INT 21H
    MOV DL, AL      ; Move input to output register
    MOV AH, 02H     ; Display character
    INT 21H
    MOV AH, 4CH     ; Exit to DOS
    INT 21H
MAIN ENDP
END MAIN
```

Example 3: Display a String

```asm
.MODEL SMALL
.STACK 100H
.DATA
MSG DB 'Bangladesh is our country$' ; $ terminates string
.CODE
MAIN PROC
    MOV AX, @DATA   ; Initialize DS
    MOV DS, AX
    LEA DX, MSG     ; Load effective address of string
    MOV AH, 09H     ; Function 09H = Display string
    INT 21H
    MOV AH, 4CH     ; Exit to DOS
    INT 21H
MAIN ENDP
END MAIN
```

Explanation:

- `DB` defines a byte or string variable.
- `$` terminates the string for DOS string output.
- `MOV AX, @DATA` → `MOV DS, AX` initializes the **Data Segment register**, required to access variables in `.DATA`.
- `LEA DX, MSG` loads the **offset address** of the string into DX.